Warehouse Shelf-Picking Agent

Abdur Rafay Ali Khan **20231-34594**

Somil **20231-34062**

Institute of Business Management

ABSTRACT

This project presents the design and simulation of an autonomous warehouse shelf-picking agent operating in a grid-based environment. The warehouse is modeled with fixed obstacles representing shelves, and a mobile robot is tasked with retrieving multiple items while minimizing time and energy costs. Path planning is performed using the A* search algorithm with a Manhattan distance heuristic to ensure collision-free and efficient navigation. Two item-picking strategies are implemented and compared: a simple single-return approach and a batch-picking strategy based on a greedy nearest-neighbor heuristic. The simulation tracks movement and pickup costs to evaluate performance. Results show that batching multiple item pickups into a single tour significantly reduces overall travel cost compared to retrieving items individually. This project demonstrates how heuristic search and basic optimization techniques can be applied to practical warehouse automation problems.

TABLE OF CONTENTS

Chapter I: Introduction *4*

Chapter II: Problem Statement *6*

Chapter III: Goals *8*

Chapter IV: Solution *10*

Chapter V: Contribution of Members *18*

Chapter VI: Conclusion *19*

CHAPTER I: INTRODUCTION

Modern warehouse operations increasingly rely on automation to improve efficiency, reduce operational costs, and minimize human error. One of the most common tasks in automated warehouses is item picking, where a mobile agent navigates through aisles of shelves to retrieve requested products and return them to a designated drop-off location. The efficiency of this process depends heavily on how well the environment is represented, how intelligently the agent plans its movements, and how effectively multiple item requests are handled.

This project focuses on the design and simulation of a warehouse shelf-picking agent within a grid-based environment. The primary objective is to model the warehouse spatially and implement decision-making strategies that allow an autonomous robot to retrieve items efficiently while accounting for time and energy costs. The agent operates in a constrained environment with fixed obstacles representing shelves and must plan collision-free paths to reach item locations.

A key challenge addressed in this project is the tradeoff between shortest-path navigation and order batching strategies. While selecting the shortest path to a single item minimizes immediate travel cost, repeatedly returning to the starting location after each pickup can lead to inefficiencies when multiple items are requested. To explore this tradeoff, the project compares a naive single-order strategy with a batch-based approach that optimizes the sequence in which multiple items are collected.

The system is implemented as an interactive simulation that allows users to place obstacles, define item locations, and observe the agent's behavior under different strategies. Path planning is handled using a heuristic search algorithm, while batching decisions are made using a greedy optimization approach. Throughout the simulation, the agent's performance is evaluated using accumulated time and energy metrics, enabling quantitative comparison between different decision-making strategies.

By combining spatial representation, pathfinding, and simple optimization techniques, this project demonstrates how fundamental artificial intelligence concepts can be applied to a practical robotics-inspired problem. The results highlight both the strengths and limitations of heuristic-based decision models in structured environments such as warehouses.

CHAPTER II: PROBLEM STATEMENT

Warehouse item picking is a fundamental task in logistics and fulfillment systems, requiring an agent to navigate a structured environment while retrieving multiple items efficiently. The core challenge lies in determining how an autonomous agent should move through a warehouse layout containing fixed obstacles (shelves) to minimize operational costs such as travel time and energy consumption.

In this project, the warehouse is modeled as a two-dimensional grid in which certain cells are designated as walkable space while others represent obstacles. A mobile robot starts from a fixed home location and is given a set of item locations that must be collected. The robot must compute valid paths that avoid obstacles and execute pickup actions upon reaching each item.

The problem becomes more complex when multiple items must be collected during a single mission. A naive approach, in which the robot returns to the starting location after each pickup, guarantees simplicity but can lead to unnecessary travel and increased cost. Alternatively, collecting items in batches requires deciding the optimal order in which items should be visited, which introduces an optimization problem that bears similarity to the Traveling Salesman Problem (TSP).

The main problem addressed in this project can therefore be summarized as follows:

- How can a warehouse environment be represented in a way that supports efficient navigation?

- How can the robot compute collision-free paths between locations within this environment?
- How do different item-picking strategies affect total time and energy consumption?
- What tradeoffs arise between shortest-path planning and order batching?

The goal is not to find a globally optimal solution to all possible warehouse configurations, but rather to design and evaluate practical heuristic-based strategies that balance computational simplicity with improved operational efficiency.

CHAPTER III: GOALS

The primary goal of this project is to design and simulate an autonomous warehouse picking agent that can efficiently retrieve items from a structured environment while accounting for movement constraints and operational costs.

More specifically, the objectives of the project are:

1. **To model a warehouse environment spatially:** Represent the warehouse as a two-dimensional grid that includes walkable areas, fixed obstacles representing shelves, item locations, and a home position for the robot.
2. **To implement a pathfinding mechanism for robot navigation:** Enable the robot to compute valid, collision-free paths between locations in the warehouse using a heuristic-based search algorithm.
3. **To explore different item-picking strategies:** Compare a simple single-order return strategy with a batch-picking strategy that optimizes the order in which multiple items are collected.
4. **To simulate time and energy costs:** Track and accumulate time and energy consumption for movement and item pickup actions to quantitatively evaluate agent performance.
5. **To analyze tradeoffs between simplicity and efficiency:** Evaluate how batching and route optimization can reduce overall cost compared to naive strategies, while also recognizing the limitations of heuristic-based approaches.

By achieving these goals, the project demonstrates how core artificial intelligence concepts such as state-space representation, heuristic search, and greedy optimization can be applied to a practical warehouse automation problem.

CHAPTER IV: SOLUTION

METHODOLOGY

The solution is based on modeling the warehouse as a grid-based environment and equipping a single autonomous agent with pathfinding and decision-making capabilities. The agent's task is to navigate through the warehouse, avoid obstacles, pick requested items, and return to the home location while minimizing time and energy costs.

Warehouse Representation

The warehouse is represented as a two-dimensional grid where each cell corresponds to a discrete location. Cells may be classified as:

- Walkable space, where the robot can move freely
- Obstacles, representing shelves that block movement
- Item locations, where pickups occur
- Home location, which serves as the robot's starting and ending point

This discrete representation simplifies path planning and allows classical graph-search algorithms to be applied directly.

Robot Movement and Cost Model

The robot is allowed to move one cell at a time in four directions: up, down, left, and right. Diagonal movement is not permitted. Each movement action incurs a fixed time cost and energy cost, while each item pickup incurs an additional fixed cost. These costs are accumulated throughout the simulation to evaluate the efficiency of different strategies.

Pathfinding Strategy

To navigate between locations in the warehouse, the agent uses the A* search algorithm. A* is a heuristic search method that guarantees the shortest path in a grid environment when an admissible heuristic is used.

In this project:

- The search space consists of all walkable grid cells.
- The heuristic function is the Manhattan distance, which is well-suited for grid-based movement without diagonals.
- Obstacles are excluded from the search, ensuring collision-free paths.

This approach allows the robot to compute optimal paths between its current position and the next target location.

Item Picking Strategies

Two different picking strategies are implemented and compared:

1. **Single-Return Strategy:** The robot retrieves one item at a time and returns to the home location after each pickup. While simple, this approach can result in redundant travel when multiple items are requested.
2. **Batch-Picking Strategy:** Multiple items are collected in a single tour before returning home. To determine the order in which items are visited, a greedy optimization approach inspired by the Traveling Salesman Problem (TSP) is used. At each step, the robot selects the nearest unvisited item based on Manhattan distance.

This batching strategy aims to reduce total travel distance and overall cost, at the expense of additional planning complexity.

Simulation Flow

The simulation proceeds as follows:

1. The user defines obstacles and item locations within the grid.
2. A picking strategy is selected.
3. The robot computes paths between successive targets using A*.
4. The robot moves step-by-step along each path, accumulating time and energy costs.
5. The simulation ends once all items are collected and the robot returns home.

This modular design separates pathfinding, decision-making, and cost evaluation, making the system easy to analyze and extend.

IMPLEMENTATION

The system is implemented in Python as an interactive simulation that combines environment representation, pathfinding, and decision-making logic. The code is structured into modular components, each responsible for a specific aspect of the warehouse agent's behavior.

Environment and Grid Setup

The warehouse environment is implemented as a grid where each cell represents a discrete location. The grid stores information about walkable spaces, obstacles, and item locations. User interaction allows obstacles and items to be placed dynamically before starting the simulation.

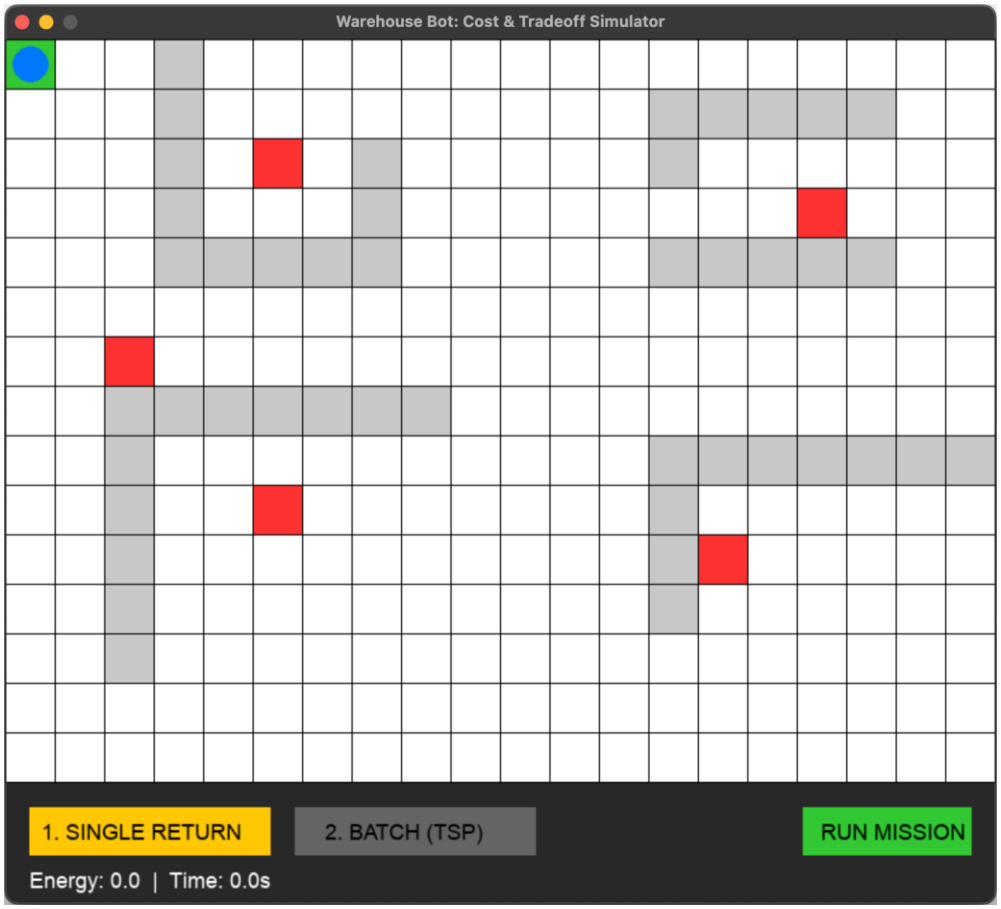

Figure 1 Grid based warehouse layout, with obstacles in gray, packages in red, the home position in green, and the robot/agent as a blue circle.

Pathfinding Implementation

Pathfinding is handled by a dedicated `PathFinder` class, which implements the A* search algorithm. The algorithm maintains a priority queue of frontier nodes, tracks the cost of reaching each cell, and uses the Manhattan distance heuristic to guide the search.

Each time the robot needs to move to a new target location, the pathfinder computes an optimal, collision-free path through the grid. The resulting path is then executed one step at a time during the simulation.

Key implementation details:

- Grid neighbors are limited to four directions (up/down/left/right).
- Obstacles are excluded from the search using walkability checks.
- Paths are reconstructed by storing parent nodes during the search.

Below we have included the code of our A* based search of the grid to obtain paths along our traversal to targets and back to the home position.

```
def get_path(self, start, goal):
    if not self.wh.is_walkable(*goal):
        return None
    frontier = []
    heapq.heappush(frontier, (0, start))
    came_from = {start: None}
    cost_so_far = {start: 0}

    while frontier:
        _, current = heapq.heappop(frontier)
        if current == goal:
            break

        r, c = current
        neighbors = [(r + 1, c), (r - 1, c), (r, c + 1),
(r, c - 1)]
        for next_node in neighbors:
            if self.wh.is_walkable(*next_node):
                new_cost = cost_so_far[current] + 1
                if (
                    next_node not in cost_so_far
                    or new_cost < cost_so_far[next_node]
                    ):
                    cost_so_far[next_node] = new_cost
                    priority = new_cost +
self.heuristic(goal, next_node)
                    heapq.heappush(frontier, (priority,
next_node))
                    came_from[next_node] = current

    if goal not in came_from:
        return None
```

```
    path = []
    curr = goal
    while curr != start:
        path.append(curr)
        curr = came_from[curr]
    path.reverse()
    return path
```

Batch Optimization Strategy

When the batch-picking mode is selected, the order in which items are collected is determined using a greedy nearest-neighbor algorithm. This logic is implemented in the `optimize_batch_route` function.

Starting from the home location, the algorithm repeatedly selects the closest unvisited item based on Manhattan distance. The resulting ordered list of item locations is then used to build the robot's target queue.

This approach provides a computationally simple approximation to the Traveling Salesman Problem and significantly reduces redundant travel compared to the single-return strategy.

Below is our TSP based path optimization code that augments our A* search to accomplish batched pickups.

```
def optimize_batch_route(self, start, orders):
    """
    Nearest Neighbor Logic (Travelling Salesman Problem)
    """
    # shallow copy orders list
    unvisited = orders[:]
    current = start
    route = []
    while unvisited:
        best_dist = float("inf")  # set to infinity
        best_node = None
        for node in unvisited:
            if node is not None:
```

```
                dist = abs(current[0] - node[0]) +
abs(current[1] - node[1])
                if dist < best_dist:
                    best_dist = dist
                    best_node = node
        if best_node is not None:
            route.append(best_node)
            unvisited.remove(best_node)
            current = best_node
        else:
            break
    return route
```

Simulation and State Management

The main simulation loop controls the robot's movement and state transitions. The robot alternates between computing paths, moving along those paths, and performing pickup actions when it reaches an item location.

During each movement step:

- Time and energy costs are incremented.
- robot's position is updated.
- Completed targets are removed from the queue.

When the robot reaches an item location, a pickup action is simulated by adding fixed pickup costs. The simulation ends once all targets have been visited and the robot returns to the home location.

Below is a snippet of our code demonstrating the simulation and calculation of the time and energy costs of each movement and item pickup action.

```
# Simulate movement energy and time costs
self.total_energy += COST_MOVE_ENERGY
self.total_time += COST_MOVE_TIME

current_target = self.target_queue[0]
if self.robot_pos == current_target:
    if current_target != (0, 0):
```

```
        # Pick item and simulate picking energy and time
cost
        self.picked_orders.append(current_target)
        self.total_energy += COST_PICK_ENERGY
        self.total_time += COST_PICK_TIME
```

Performance Tracking and Output

Throughout the simulation, the system tracks total time, total energy consumption, and the number of items picked. These metrics are displayed at the end of the run, allowing direct comparison between different picking strategies.

This implementation makes it possible to observe how pathfinding and batching decisions affect overall efficiency, providing a practical demonstration of decision modeling in a spatial environment.

CHAPTER V: CONTRIBUTION OF MEMBERS

This project was completed as a collaborative effort between two group members. Both members contributed to the overall design, implementation, and documentation of the warehouse picking agent.

One member primarily focused on the design and implementation of the simulation framework, including the warehouse grid representation, user interaction, and the main simulation loop. This included handling robot movement, state transitions, and the accumulation of time and energy costs.

The other member concentrated on algorithmic components, particularly the implementation of the pathfinding logic using the A* search algorithm and the batch optimization strategy for item picking. This member also contributed to evaluating and comparing different picking strategies.

Both members jointly participated in testing, debugging, and refining the system, as well as preparing the written report and analyzing the results.

CHAPTER VI: CONCLUSION

In this project, a warehouse shelf-picking agent was designed and simulated to explore the interaction between spatial representation, pathfinding, and decision-making strategies. The warehouse environment was modeled as a grid with obstacles and item locations, allowing classical search algorithms to be applied to a practical automation problem.

The use of the A* algorithm enabled the robot to compute optimal, collision-free paths within the warehouse, ensuring efficient navigation between targets. Two item-picking strategies were implemented and compared: a simple single-return approach and a batch-picking strategy based on a greedy nearest-neighbor heuristic. The results demonstrate that batching multiple items into a single tour can significantly reduce total travel time and energy consumption compared to retrieving items individually.

While the batch-picking strategy improves efficiency, it relies on heuristic approximations and does not guarantee optimal solutions, particularly in environments with complex obstacle layouts. Nonetheless, the approach provides a good balance between performance and computational simplicity.

Overall, this project illustrates how fundamental artificial intelligence techniques such as heuristic search and greedy optimization can be combined to address real-world inspired problems in warehouse automation. The system also provides a foundation for future extensions, such as more advanced optimization algorithms, dynamic replanning, or multi-agent coordination.